SENDA2 y el acomodo tridimensional: del ruteo clásico al problema integrado, medido con honestidad

Segunda guía educativa del motor LATTIMEX (versión 1.1.0): qué cambió, cómo se midió y cómo probarlo en su computadora

Erick Martínez
LATTIMEX · investigación independiente · Querétaro, México
contacto@lattimex.com · https://lattimex.com

Octubre de 2026

Resumen

En pocas palabras. La primera guía abrió SENDA, el núcleo de ruteo de LATTIMEX, y lo midió en el problema clásico. Esta segunda guía cuenta el avance: un núcleo mejor (SENDA2), un acomodador tridimensional nuevo (P3L) y su integración en el motor 1.1.0, que es de código abierto (MIT). Igual que la primera, explica cada pieza desde cero y dedica tanto espacio a *cómo* se midió como a los resultados, incluidos los errores propios que la medición destapó.

SENDA2 agrega a SENDA tres cambios en su búsqueda adaptativa de vecindarios grandes: una reparación con caché, el descarte de reparaciones que no caben y un tope de 0.02 s al pulido de cada nuevo mejor. En una misma sesión, con el mismo reloj y verificación independiente, queda a +0.33 pp de HGS-CVRP en 41 instancias X a 4 s (1.74 % frente a 1.40 %; FILO2, 0.72 %) y a +0.07 pp en A/B/E a 2 s con flota fija; la primera guía reportaba una distancia de unos 2.6 pp en X. Para la carga se construyó un arnés contra los óptimos publicados de Tamke et al. (2026) y el validador de Krebs y Ehmke (2023). Con él se descubrió que el empacador de la versión 1.0.0 rechazaba entre 34 % y 61 % de rutas que sí caben y que permitía una caja posterior encima de una anterior. P3L baja ese rechazo a 24–36 % y, con el verificador estricto, el motor ya no entrega acomodos que violen el orden de descarga. Un ruteo que revisa la carga en cada paso (R3L) logra 285 de 285 soluciones válidas en las 19 instancias de Gendreau, a +6.9 % de distancia de las soluciones de Tamke (que corrió hasta 8 horas por instancia). R3L no escala a 300 entregas; el documento explica por qué y qué queda pendiente.

Palabras clave: ruteo de vehículos; CVRP; 3L-CVRP; ALNS; acomodo tridimensional; orden de descarga (LIFO); evaluación reproducible; docencia en optimización.

Índice

Cómo leer este documento **3**

1. Introducción **3**

2. SENDA2: el núcleo de ruteo **4**
2.1. Qué cambia respecto de SENDA 4
2.2. Cómo se llegó: dos errores útiles 5
2.3. Resultados frente a HGS-CVRP y FILO2 5
2.4. SENDA2 dentro del motor 6

© 2026 Erick Martínez (LATTIMEX). CC BY 4.0: https://creativecommons.org/licenses/by/4.0/

3. La carga: medir y construir un acomodador **7**
3.1. El problema con carga tridimensional . . . 7
3.2. Cómo medir un acomodador sin engañarse . . . 8
3.3. Lo que encontró la medición en la versión 1.0.0 . . . 8
3.4. P3L . . . 8

4. La integración: ruteo que sabe lo que cabe **9**
4.1. Por qué un buen acomodador no basta . . . 9
4.2. R3L . . . 9
4.3. Resultados contra Tamke . . . 10
4.4. A escala de producto . . . 11

5. Discusión **11**

6. Conclusiones y trabajo futuro **12**

7. Pruébelo en su computadora **12**

Cómo leer este documento

Esta guía continúa a la primera (*SENDA: ruteo de vehículos con búsqueda de trayectoria única, explicado y medido con honestidad*, 30 de septiembre de 2026, `paper/guia_senda.pdf`). No la reemplaza: la primera sigue siendo la referencia para entender SENDA pieza por pieza (construcción, búsqueda local granular, ALNS, movimientos de segmento, portafolio) y para las bases de una comparación justa. Aquí se supone ese conocimiento y se explica lo nuevo.

- La Sección 2 presenta SENDA2, cómo se llegó a él y cómo se compara con HGS-CVRP y FILO2 en el problema clásico.
- La Sección 3 introduce el problema con carga tridimensional, explica cómo medir un acomodador sin engañarse y presenta P3L.
- La Sección 4 une ruteo y acomodo: por qué un buen acomodador no basta, qué es R3L y qué logra el motor 1.1.0.
- La Sección 7 explica cómo usar el motor completo o cada pieza por separado.

Los recuadros tienen las mismas cuatro funciones que en la primera guía:

Idea clave

Resume el concepto que no debe olvidarse.

Ejemplo resuelto

Resuelve un cálculo pequeño, paso a paso y con números concretos.

Lección metodológica

Describe un error posible, o uno que realmente ocurrió en este proyecto, y cómo evitarlo.

Para reproducir

Indica las huellas SHA-256 de los artefactos que respaldan una cifra y cómo repetir la medición.

Sobre esta edición. Acompaña a la versión 1.1.0 del repositorio público de LATTIMEX. El repositorio contiene el motor y esta guía; no contiene los programas de comparación (HGS-CVRP, FILO2, el validador de Krebs, el modelo exacto con CP-SAT) ni los scripts de las campañas. Se describen aquí para que la comparación sea transparente; los datos crudos, contratos y bitácoras se conservan en el archivo del autor y cada cifra cita la huella del artefacto que la respalda.

1. Introducción

La primera guía terminaba con tres tareas pendientes: dar más tiempo efectivo a la búsqueda en instancias grandes, medir el núcleo en el problema integrado con carga tridimensional y ampliar el banco de pruebas. Esta guía cuenta lo que pasó con las dos primeras.

La respuesta corta tiene tres partes:

1. **SENDA2.** Tres cambios pequeños al ALNS de SENDA cierran la mayor parte de la distancia con HGS-CVRP en las instancias X: de unos 2.6 a 0.33 puntos de gap a 4 s. El camino incluyó un error de medición propio y una hipótesis (la semilla *k-medoids*) que la ablación descartó.

2. **P3L y el verificador estricto.** Medir el empacador de la versión 1.0.0 contra rutas que *se sabe* que caben reveló que rechazaba la mitad de ellas y que su regla de orden de descarga tenía un hueco. P3L es un acomodador nuevo con la semántica estándar; el verificador estricto garantiza que el motor no entregue un acomodo inválido.

3. **La integración.** Un buen acomodador no basta: si el ruteo no sabe qué cabe, propone rutas imposibles. R3L, un ruteo que consulta al acomodador en cada paso, resuelve la validez en instancias pequeñas; a escala de producto el motor usa P3L dentro del ruteo de SENDA2 y avisa cuando no logra respetar el orden de descarga.

Idea clave

Cada avance de esta guía empezó por una medición que contradijo lo que se creía. La versión 1.0.0 no era peor de lo anunciado en ruteo, pero sí en carga: con orden de descarga entregaba planes que lo violaban. Medir contra óptimos conocidos y con un juez independiente fue lo que lo hizo visible.

2. SENDA2: el núcleo de ruteo

2.1. Qué cambia respecto de SENDA

SENDA (primera guía, Secciones 3 a 7) sigue una sola trayectoria por corrida: construye, mejora con búsqueda local granular, reestructura con un ALNS que destruye parte de la solución y la repara, pule cada nuevo mejor y reparte el tiempo entre dos o tres configuraciones. SENDA2 conserva todo eso y cambia tres cosas dentro del ALNS:

(a) **Reparación regret con caché.** Al reinsertar los clientes quitados, la reparación de SENDA recalcula, para cada cliente y en cada paso, su mejor posición en todas las rutas. SENDA2 guarda una sola inserción candidata por ruta para cada cliente (la más barata entre las posiciones contiguas a sus vecinos granulares) y, después de cada inserción, sólo recalcula la ruta que cambió. Cada reparación cuesta mucho menos y el ALNS hace más iteraciones en el mismo tiempo.

(b) **Descarte de reparaciones fallidas.** Con flota fija y sin penalización, si la reparación no logra meter a todos los clientes sin exceder la capacidad, SENDA2 descarta el candidato (y castiga los pesos de los operadores que lo produjeron) en lugar de aceptarlo infactible.

(c) **Tope de pulido.** Cada vez que aparece un nuevo mejor, SENDA lo pule con búsqueda local hasta que no mejora. SENDA2 limita ese pulido a $\min(0.02\,\text{s}, 0.20 \cdot t_{\text{rest}})$, donde t_{rest} es el tiempo que queda.

Ejemplo resuelto: cuánto tiempo libera el tope de pulido

En X a 4 s, cada solver dispone del 80 % del reloj: 3.2 s. En la medición previa a SENDA2, el pulido de cada nuevo mejor consumía en promedio alrededor de 1.9 de esos 3.2 s, casi el 60 %. Con el tope de 0.02 s, un ensayo que encuentra 40 nuevos mejores gasta a lo sumo $40 \times 0.02 = 0.8$ s en pulir, y el resto vuelve al ALNS. En la confirmación (Sección 2.3), las iteraciones medias en X pasaron de 654 a 1 402 por ensayo y el gap bajó 0.59 puntos.

2.2. Cómo se llegó: dos errores útiles

Lección metodológica: el control degradado

El primer candidato, llamado *KM-R1*, sembraba la solución inicial con k-medoides y destruía por medoides. Su primera comparación contra SENDA mostró una ventaja grande. Al repetirla se descubrió que el control (SENDA) había corrido a la mitad de sus iteraciones normales: otra sesión, otra carga de la máquina. Repetida con los tres brazos en la misma sesión y con un *sentinela de velocidad* (las iteraciones del control deben coincidir con las de una corrida de referencia), la ventaja seguía siendo real pero menor. Regla: **todos los brazos en la misma sesión, con orden rotado y un sentinela de velocidad**; nunca contra un número de otra sesión.

Lección metodológica: el nombre que la ablación quitó

Con la ventaja confirmada, era tentador atribuirla a los k-medoides. La ablación (quitar una pieza a la vez, en la misma sesión) dijo otra cosa: quitar la semilla o la destrucción por medoides no cambiaba nada; quitar la reparación con caché costaba 0.87 puntos en X, y el tope de pulido explicaba la mayor parte del resto. El candidato se simplificó a esas piezas y perdió el nombre "SENDA-KM": se llama SENDA2. Además, el tope de 0.02 s se eligió mirando los datos de una ronda, así que su cifra no podía publicarse sin repetirla con semillas nuevas (Sección 2.3).

2.3. Resultados frente a HGS-CVRP y FILO2

Diseño. Igual que en la primera guía: 60 instancias A/B/E con flota fija a 2 s y 41 instancias X con flota libre a 4 s; el 80 % del reloj como presupuesto interno para todos los solvers; un hilo; distancias redondeadas al entero (convención de CVRPLIB); verificación independiente de cobertura, capacidad, flota y costo. HGS-CVRP [9] y FILO2 [1] se compilaron de sus fuentes oficiales. La tabla principal es una sesión con los tres solvers y semillas nunca usadas (3000–3900); la confirmación H1-C, con otras semillas nuevas (2000–2900), repite la comparación con HGS.

Tabla 1: Gap medio contra la mejor solución conocida (BKS). Misma sesión, mismo reloj. En A/B/E se promedian las 55 instancias donde los tres solvers respetaron la flota en todas sus corridas (FILO2 no tiene opción de flota fija y usó una ruta de más en 5 instancias).

	SENDA2	HGS-CVRP	FILO2
X, 41 instancias, 4 s, 5 semillas	1.74 %	1.40 %	**0.72 %**
A/B/E, 55 instancias, 2 s, 10 semillas	0.074 %	**0.006 %**	0.053 %

Comparaciones pareadas. Por instancia, Δ = gap de SENDA2 menos gap del otro solver (IC95 por bootstrap; prueba de Wilcoxon):

- **Contra HGS, en X:** +0.33 pp [0.21, 0.45], $p = 3 \cdot 10^{-6}$; SENDA2 es mejor en 7 de 41 instancias. La confirmación H1-C da +0.34 pp con SENDA2 y +0.28 pp [0.13, 0.42] con una variante que conservaba las piezas descartadas.
- **Contra HGS, en A/B/E:** +0.068 pp [0.033, 0.111]; empata en 36 instancias y HGS es mejor en 19.
- **Contra FILO2:** en X, +1.02 pp [0.85, 1.17], y FILO2 gana en 40 de 41; en A/B/E no se detectó una diferencia clara (+0.021 pp, IC que cruza cero).

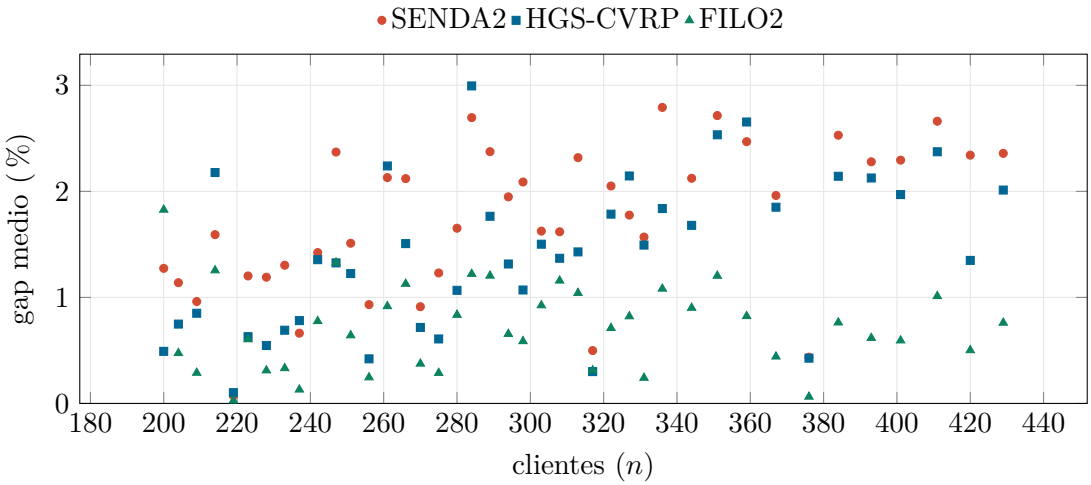


Figura 1: Gap medio por instancia X a 4 s (5 semillas por solver, misma sesión). La ventaja de FILO2 crece con el tamaño de la instancia.

Contra SENDA. La primera guía reportaba, en otra máquina, 4.119 % en X frente a 1.450 % de HGS. Para no mezclar sesiones, la comparación honesta usa a HGS como ancla común: en una sesión de octubre, SENDA quedaba a +2.64 pp de HGS en X (4.05 % frente a 1.41 %); SENDA2 queda a +0.33 pp. En A/B/E, de +0.094 a +0.068 pp.

Idea clave

La afirmación honesta no es "estado del arte": FILO2 es claramente más fuerte en instancias grandes con reloj corto, y HGS sigue siendo mejor en promedio. SENDA2 es un núcleo de unas 2 200 líneas, de una sola trayectoria, que respeta la flota fija y queda cerca de HGS. Su valor está en ser simple, auditable y medido con cuidado.

Para reproducir

Sesión con los tres solvers: `raw_trials.jsonl ef823740c317...`, contrato `906e37b0ab29...`. Confirmación H1-C: `2ac424ad3411...`. Con el repositorio: `python -m lattimex cvrp INSTANCIA.vrp -time 4` (X) o `-time 2 -fleet fixed` (A/B/E) reparte el tiempo y elige configuraciones igual que en la medición. Repetido sobre el mismo banco con el binario publicado, una corrida a la vez, dio 1.77 % en X (41 instancias, 1 semilla) y 0.086 % en A/B/E (60 instancias, 2 semillas), dentro de la variación entre semillas. **Corra una instancia a la vez**: con seis procesos en paralelo en la misma máquina el gap en X subió a 1.96 %, porque cada proceso hizo menos iteraciones.

2.4. SENDA2 dentro del motor

El motor no llama al núcleo igual que el banco: reparte el presupuesto entre ruteo, acomodo y reoptimización, usa flota exacta y corre un portafolio de configuraciones en dos hilos. Medido así (107 instancias, 3 semillas, misma sesión), SENDA2 contra SENDA:

- A/B/E: −0.036 % de distancia [−0.068, −0.009], $p = 0.02$.
- X: el resultado depende de cuántos clientes atiende cada ruta (Spearman $\rho = -0.65$). Con 12 o más clientes por ruta, −0.89 % (gana en 14 de 16); con menos de 6, +2.99 % (pierde en 9 de 12).

Las rutas reales de LATTIMEX son largas (de 30 a 75 entregas por unidad), así que el motor 1.1.0 usa SENDA2 por omisión y vuelve al núcleo de 1.0.0 cuando hay menos de 6 entregas por

unidad. La regla se fijó con los mismos datos que la motivan; es una decisión de ingeniería, no una cifra confirmada.

La misma ronda encontró un riesgo del constructor de flota exacta: cuando la carga ocupa entre 92.5 % y 99.7 % de la capacidad total, no lograba repartirla en k unidades y el motor rechazaba el pedido (11 de 41 instancias X). El motor 1.1.0 agrega un respaldo que reparte aceptando sobrecarga y la elimina con búsqueda local; resolvió las 11 en menos de medio segundo.

3. La carga: medir y construir un acomodador

3.1. El problema con carga tridimensional

En el 3L-CVRP [3] cada cliente pide cajas con largo, ancho, alto, peso y una marca de fragilidad, y cada ruta debe poder acomodarse dentro de la caja del vehículo. Se usan los ejes de Gendreau: x es el largo (la puerta está en $x = L$), y el ancho y z el alto; las cajas sólo rotan en horizontal. Además de no salirse ni encimarse, un acomodo puede exigir:

- **Soporte:** al menos el 75 % de la base de cada caja descansa sobre el piso o sobre cajas cuya cara superior está a esa altura.
- **Fragilidad:** una caja no frágil no puede descansar sobre una frágil.
- **Orden de descarga (LIFO):** al llegar con cada cliente, sus cajas salen sin mover las de clientes posteriores.

Las cinco variantes estándar combinan estas reglas:

Tabla 2: Variantes del 3L-CVRP (Gendreau et al., 2006).

Variante	Soporte	Fragilidad	Orden de descarga
loading-only	–	–	–
no-support	–	sí	sí
no-fragility	sí	–	sí
no-lifo	sí	sí	–
all-constraints	sí	sí	sí

El orden de descarga, con precisión. Sea e una caja de un cliente que se visita *antes* que el de la caja t. El acomodo viola el orden si t bloquea a e de alguna de dos formas (Figura 2): (i) t queda *delante* de e, más cerca de la puerta, y sus proyecciones sobre el plano y–z se tocan; o (ii) t queda *encima* de e y sus proyecciones sobre el plano x–y se tocan. Ésta es la semántica del validador público de Krebs y Ehmke [6, 5].

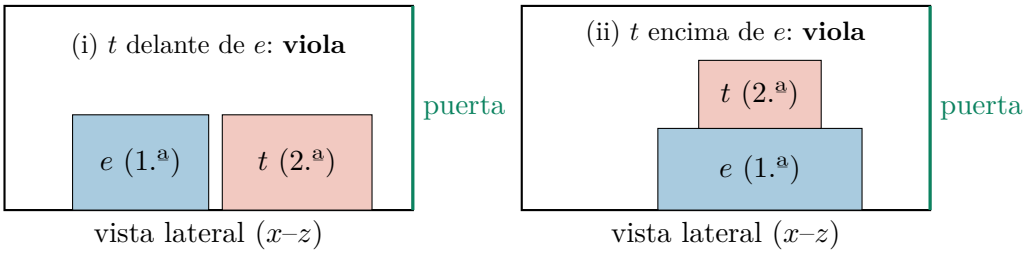


Figura 2: Las dos formas en que una caja t de una entrega posterior bloquea a una caja e de una entrega anterior. El empacador de la versión 1.0.0 sólo prohibía la primera.

3.2. Cómo medir un acomodador sin engañarse

Medir un acomodador parece fácil: se le dan rutas y se cuenta cuántas acomoda. La trampa es que no se sabe cuántas *deberían* caber. Si se le dan rutas holgadas, cualquier acomodador parece bueno; si se le dan rutas imposibles, cualquiera parece malo.

Rutas que se sabe que caben. Tamke et al. [8] resolvieron con un *branch-and-cut* exacto las 19 instancias clásicas de Gendreau con hasta 50 clientes, en las cinco variantes, y publicaron sus soluciones con el acomodo de cada ruta [7]. Cada una de esas 648 rutas cabe: su acomodo existe y el validador lo aprueba. Si un acomodador rechaza una de ellas, es un **falso rechazo**, sin ambigüedad. A esta prueba se le llama *nivel 1*.

Un juez independiente. Todo acomodo aceptado se revisa con el validador de Krebs, que no forma parte del motor. Antes de usarlo como juez se comprobó que juzga bien: aprueba las 95 soluciones de Tamke (19 instancias × 5 variantes), rechaza las 76 soluciones de variantes laxas evaluadas con todas las restricciones y rechaza los 19 casos con un solape forzado.

Lección metodológica: el 2.4 % que no medía lo que parecía

En julio se había reportado que el empacador del motor rechazaba sólo el 2.4 % de las rutas. Esa medición usaba rutas de otra heurística construidas con todas las restricciones y las probaba en la variante más laxa: eran rutas holgadas. Con las rutas óptimas, que están apretadas, el rechazo era del 34 al 64 %. Una cifra de calidad sólo significa algo junto con el banco en que se midió.

3.3. Lo que encontró la medición en la versión 1.0.0

- **Falso rechazo alto:** entre 36 % (loading-only) y 64 % (all-constraints) de las rutas óptimas, y crece con el tamaño de la ruta: con 9 a 12 cajas, 47 de 49 rechazadas. Con 1 024 reinicios en lugar de los habituales, all-constraints sólo bajó de 64 % a 60 %: el límite era el algoritmo de colocación, no el esfuerzo.
- **Un hueco en el orden de descarga:** el empacador prohibía el caso (i) de la Figura 2 pero no el (ii). Krebs rechazó 27 acomodos que el motor daba por buenos.
- **En el motor completo,** con todas las restricciones, 1 de 57 corridas fue válida: cuando una ruta no cabía, el motor relajaba el orden de descarga en silencio.

3.4. P3L

P3L (`native/p3l.cpp`) acomoda una ruta en un contenedor con la semántica de la Figura 2:

1. **Secuencia de carga.** Con orden de descarga, las cajas se cargan del último cliente al primero; dentro de cada cliente, por volumen, área de base, altura o lado mayor.
2. **Colocación por gravedad.** Para cada caja se generan posiciones candidatas en x y y (el origen, el final de cada caja ya puesta y la pared opuesta) y cada una baja hasta descansar; gana la primera según una regla lexicográfica (por ejemplo, menor x, luego menor z).
3. **Búsqueda en haz** sobre la secuencia: se conservan las mejores secuencias parciales según el contacto con paredes y cajas.
4. **Búsqueda local iterada** sobre la secuencia de carga y las rotaciones, desde varios arranques, hasta colocar todo o agotar el tope (50 ms por ruta en el motor).

Lección metodológica: posiciones que parecen bastar y no bastan

Para acelerar, es común restringir las posiciones a *patrones normales*: las sumas de longitudes de cajas. Es correcto sin soporte ni orden de descarga, pero no con ellos. Un modelo exacto con CP-SAT, usado como referencia durante el desarrollo, "demostró" imposibles 5 rutas que Tamke había acomodado. La causa eran los patrones normales; con todas las posiciones enteras en x y y cuando hay soporte, como hace Tamke, las 5 rutas cupieron.

Resultados en el nivel 1. Las 13 instancias que no se usaron para ajustar P3L, con el candidato congelado (Tabla 3). La columna "P3L + CP-SAT" agrega el modelo exacto para las rutas que P3L no acomoda; baja el rechazo casi a cero, pero no forma parte del motor: depende de OR-Tools y no termina con rutas de 55 o más cajas.

Tabla 3: Falso rechazo sobre las rutas óptimas de Tamke, 13 instancias de confirmación. Ningún acomodo aceptado por P3L fue rechazado por el validador.

Variante	Empacador 1.0.0	**P3L** (motor 1.1.0)	P3L + CP-SAT
loading-only	34.1 %	23.9 %	1.1 %
no-support	42.2 %	33.3 %	0.0 %
no-fragility	57.6 %	32.6 %	0.0 %
no-lifo	41.1 %	35.6 %	6.7 %
all-constraints	61.3 %	30.1 %	0.0 %

A escala de producto. En 159 rutas de 20 a 97 cajas producidas por el motor en 12 instancias de Querétaro, con todas las restricciones, el empacador de 1.0.0 rechazó al menos el 70.8 % de las rutas que caben y dio 9 acomodos inválidos; P3L con 50 ms, al menos el 14.6 %, y con 500 ms, al menos el 3.8 %, sin ningún inválido ("al menos" porque aquí no hay óptimos: una ruta cuenta como factible si algún método la acomodó). Sin orden de descarga, ambos acomodan todas.

El verificador estricto. El motor 1.1.0 revisa todo acomodo con la semántica de la Figura 2 antes de aceptarlo, venga de P3L o del empacador de 1.0.0, que se conserva como respaldo porque puede girar en vertical las cajas que lo permiten. Contra el validador de Krebs, el verificador coincidió en 2 436 de 2 436 acomodos, incluidos 105 que Krebs rechaza.

4. La integración: ruteo que sabe lo que cabe

4.1. Por qué un buen acomodador no basta

El motor rutea con una capacidad de volumen efectiva, una fracción del volumen del vehículo, y acomoda después. Si una ruta no cabe, una *mediación* prueba unos pocos cambios: invertirla, rotarla o mover un cliente a otra ruta. Al sustituir sólo el acomodador por P3L, las corridas válidas con todas las restricciones subieron de 1 a 20 de 57, pero el gap de las válidas no cambió (+15 a +29 %) y el modelo exacto demostró imposibles 1 665 rutas que el ruteo había propuesto. El ruteo seguía proponiendo rutas sin saber si cabían.

4.2. R3L

R3L (`lattimex/engine/r3l.py`) es un ruteo de destruir y reconstruir al estilo SISR [2] con recocido simulado [4], en el que cada inserción consulta al acomodador:

Algoritmo 1 R3L (esquema)

Entrada: clientes, matriz de distancias, contenedor, variante, flota máxima k, tiempo T

```
S ← reconstruir desde cero; si hay rutas de SENDA2, también desde ellas, y quedarse con la mejor
mientras quede tiempo hacer
    S′ ← quitar de S cadenas de clientes cercanos (SISR)
    para cada cliente quitado, en un orden al azar o por volumen o distancia hacer
        probar sus posiciones de la más barata a la más cara;
        quedarse con la primera cuya ruta cabe (P3L); abrir ruta nueva sólo si hay menos de k
    fin para
    aceptar S′ con el criterio de recocido simulado; guardar la mejor solución completa
fin mientras
```

Tres detalles lo hacen rápido. (1) La revisión se escalona: una pasada voraz de P3L para cada posición candidata (0.3 ms con rutas pequeñas) y la búsqueda en haz sólo para la posición más barata. (2) Un caché guarda cada veredicto; sin orden de descarga la llave es el conjunto de clientes, porque el orden no cambia el acomodo. (3) Con orden de descarga, si una ruta no cabe se prueba invertida: tiene la misma distancia y otro orden de descarga.

Ejemplo resuelto: la ruta invertida

Una ruta visita a A y luego a B. A pide una caja no frágil que ocupa todo el piso; B, una frágil del mismo tamaño. En el orden A, B, la caja de B (segunda entrega) debe cargarse primero, al fondo o abajo; como ocupa todo el piso, queda abajo y la de A encima: una caja no frágil sobre una frágil, prohibido. En el orden B, A, la de A va abajo y la frágil de B encima: permitido, y B sale primero sin mover nada. La distancia es la misma; R3L sólo tiene que probar la ruta en ambos sentidos.

4.3. Resultados contra Tamke

Las 19 instancias de Gendreau, las cinco variantes y tres semillas (285 corridas por brazo), con flota "hasta k" (k = vehículos disponibles, igual que Tamke). Una corrida es *válida* si el validador de Krebs aprueba ruteo y carga en la variante. El gap es la distancia contra la solución de Tamke.

Tabla 4: Motor completo contra Tamke. "Fáciles": instancias que Tamke resolvió al óptimo en menos de 1 000 s; en las demás, Tamke corrió hasta 8 horas y no cerró su propia cota.

	válidas	gap medio	gap fáciles	gap demás
Motor 1.0.0 (10 s, flota exacta)	81/285	+26.4 %	–	–
SENDA2 + R3L, investigación (10 s)	283/285	+8.7 %	+4.2 %	+13.5 %
SENDA2 + R3L, investigación (30 s)	285/285	+6.9 %	+3.7 %	+10.2 %
Motor 1.1.0, flota exacta (10 s)	145/285	+26.4 %	–	–
Motor 1.1.0, `fleet_mode=up_to` (10 s)	284/285	+8.1 %	+4.1 %	+12.3 %

Con flota exacta (la opción por omisión), el motor 1.1.0 no usa R3L: rutea con SENDA2 y acomoda con P3L. Casi duplica las corridas válidas del motor 1.0.0, pero el gap sigue alto, porque obliga a usar todas las unidades disponibles y Tamke usa sólo las necesarias. Cuando relajó el orden de descarga, el motor lo informó en la respuesta; el validador sólo rechazó corridas así.

Lección metodológica: una meta que no era realista

La meta inicial era 100 % de corridas válidas y gap ≤ 3 % en 10 s. Los tiempos de Tamke muestran que no era razonable: en las instancias difíciles, su método exacto corrió 8 horas sin demostrar el óptimo. La meta se ajustó después de ver los primeros resultados a ≤ 3 % en las instancias fáciles, y se reporta aquí junto con la original. En las fáciles el promedio es de +3.7 % a 30 s (+3.3 % en las 13 instancias que no se usaron para ajustar R3L): todavía no se cumple.

4.4. A escala de producto

R3L no escala: cada revisión vuelve a acomodar la ruta completa, y con rutas de 60 a 75 cajas una revisión cuesta de 15 a 300 ms, en lugar de los 0.3 ms de las rutas de Gendreau. Con 300 entregas, R3L no terminaba ni la solución inicial en el tiempo disponible. Por eso el motor 1.1.0 sólo lo usa con flota "hasta k" en instancias pequeñas (hasta 60 entregas y 30 cajas por unidad).

Para instancias grandes, el motor 1.1.0 rutea con SENDA2, acomoda con P3L en la variante pedida, media con el orden de descarga estricto y, sólo si una ruta sigue sin caber, la acomoda sin orden de descarga **y lo informa** en la respuesta (`lifo_estricto`, `advertencias`). En las 12 instancias de producto con todas las restricciones (Tabla 5), la versión 1.0.0 declaraba factibles todas las corridas y ninguna respetaba el orden de descarga.

Tabla 5: 12 instancias de producto (60 a 300 entregas, 1 a 10 unidades), flota exacta, 30 s, 2 semillas. "Con orden estricto": el motor la declara factible, no relajó ninguna ruta y el validador aprueba la carga.

	Motor 1.0.0	**Motor 1.1.0**
all-constraints: con orden estricto	0/24	19/24
all-constraints: relajadas e *informadas*	–	5/24
loading-only: válidas	24/24	24/24

Lección metodológica: "factible" no es "correcto"

La versión 1.0.0 decía "factible" a planes que violaban el orden de descarga, porque lo relajaba sin avisar para no partir rutas. Era una decisión de diseño razonable, pero invisible para quien leía la respuesta. La 1.1.0 mantiene la posibilidad de relajar como último recurso y la hace visible: un plan que obliga al operador a mover cajas no debe presentarse igual que uno que no.

5. Discusión

Dónde está el valor. En el problema clásico, SENDA2 acerca a LATTIMEX a HGS sin volverlo más complejo. Pero dentro del motor, con carga tridimensional, la diferencia entre núcleos de ruteo se pierde (la ronda 2 no detectó diferencia de distancia con acomodo 3D). La ganancia grande para un operador está en la carga: que el plan respete el orden de descarga o diga cuándo no puede.

Limitaciones. (i) El nivel 1 y el nivel 2 usan las 19 instancias de Gendreau con hasta 50 clientes; a escala de producto no hay óptimos y las cifras son cotas. (ii) P3L sólo gira cajas en horizontal; el empacador de 1.0.0 se conserva como respaldo para cajas que pueden girar en vertical, sin una medición propia de ese caso. (iii) La regla que elige entre SENDA y SENDA2 se fijó con los mismos datos que la motivan. (iv) R3L se ajustó con 6 instancias y se reporta también en las 13 restantes, pero las cifras de la Tabla 4 mezclan ambos grupos. (v) Todas las mediciones son de una sola máquina.

6. Conclusiones y trabajo futuro

SENDA2 cierra la mayor parte de la distancia con HGS en el CVRP clásico con tres cambios pequeños y medibles. P3L y el verificador estricto corrigen un defecto real de la versión 1.0.0 y reducen a la mitad el rechazo de rutas que caben. R3L muestra que revisar la carga dentro del ruteo resuelve la validez en instancias pequeñas.

Trabajo futuro. (1) Un acomodo *incremental*: al insertar un cliente como primera entrega, sus cajas son las últimas en cargarse y podrían añadirse sin reacomodar la ruta; es la pieza que falta para que R3L escale. (2) Un acomodador rápido más fuerte en rutas apretadas, para cerrar la meta de ≤ 3 % en las instancias fáciles. (3) Localizar la búsqueda local de SENDA2 en los vértices tocados, como FILO2, la diferencia de diseño más probable detrás de la brecha en X.

7. Pruébelo en su computadora

El motor 1.1.0 está publicado con licencia MIT en https://github.com/Erick-MF28/lattimex; la versión 1.0.0 sigue disponible en su etiqueta. Se puede usar completo o por piezas:

```
git clone https://github.com/Erick-MF28/lattimex.git
cd lattimex
pip install -r requirements.txt  # Windows sin compilador: pip install ziglang
python -m lattimex build         # compila los núcleos y los acomodadores
python -m lattimex probe         # huella determinista
python -m unittest discover -s tests/engine
```

1. SENDA2 solo, en CVRPLIB.

```
python -m lattimex cvrp X-n101-k25.vrp --time 4 --bks 27591
python -m lattimex cvrp A-n32-k5.vrp --time 2 --fleet fixed --sol A-n32-k5.sol
```

`-router senda` usa el núcleo de la versión 1.0.0, para comparar ambos en su máquina.

2. El acomodador solo. Un archivo JSON describe el contenedor y las cajas de una ruta, con `stop` = posición de entrega (1 = primera):

```
{"cargo_cm": [300, 160, 160],
 "constraints": {"support": true, "fragility": true, "lifo": true},
 "items": [{"id": "A1", "dims_cm": [60, 40, 30], "stop": 1, "fragile": false},
           {"id": "B1", "dims_cm": [40, 30, 30], "stop": 2, "fragile": true}]}
```

`python -m lattimex pack cajas.json` devuelve la posición de cada caja y si el acomodo cumple la variante.

3. El motor completo. `python -m lattimex serve` y el LATTIMEX Planner en https://lattimex.com/planner, igual que en la versión 1.0.0: el Planner usa el motor 1.1.0 sin cambiar nada. Por API, el campo `fleet_mode` con valor `up_to` permite usar menos unidades (y activa R3L en instancias pequeñas), y el campo `router` con valor `senda` fuerza el núcleo de 1.0.0.

4. Repetir la medición de carga. Las instancias y soluciones de Tamke [7] (MIT) y el validador de Krebs [5] (GPL-3.0) se obtienen de sus repositorios; no forman parte de LATTIMEX. El validador lee las restricciones por número de línea: genere los cinco archivos de variante a partir de su plantilla. Las soluciones de Tamke repiten el identificador de tour, que hay que renumerar antes de validarlas.

Para reproducir: artefactos

SENDA2, HGS y FILO2	`raw_trials.jsonl` `ef823740c317...`; contrato `906e37b0ab29...`
Confirmación H1-C	`raw_trials.jsonl` `2ac424ad3411...`
SENDA2 en el motor (ronda 2)	`summary.json` `8dfb1d575211...`
Nivel 1 (empacador 1.0.0 y P3L)	`level1.jsonl` `1944dc5be91d...`; `level1_hyb_confirm.jsonl` `e2bde2f206bd...`
Integración (investigación)	`level2_s2r3l_summary.json` `ab706daaaabf...`
Motor 1.1.0 publicado	`resumen.json` `12a78d4a782d...`
Figuras de este documento	datos en `paper/datos2/` de este repositorio
Artefactos completos	archivo del autor; se comparten a solicitud (`contacto@lattimex.com`)

Referencias

[1] Luca Accorsi and Daniele Vigo. Routing one million customers in a handful of minutes. arXiv:2306.14205, 2023.

[2] Jan Christiaens and Greet Vanden Berghe. Slack induction by string removals for vehicle routing problems. *Transportation Science*, 54(2):417–433, 2020.

[3] Michel Gendreau, Manuel Iori, Gilbert Laporte, and Silvano Martello. A tabu search algorithm for a routing and container loading problem. *Transportation Science*, 40(3):342–350, 2006.

[4] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[5] Corinna Krebs. SolutionValidator. https://github.com/CorinnaKrebs/SolutionValidator. GPL-3.0, commit 6a3b23f.

[6] Corinna Krebs and Jan Fabian Ehmke. Solution validator and visualizer for (combined) vehicle routing and container loading problems. *Annals of Operations Research*, 326:561–579, 2023.

[7] Felix Tamke. 3L-CVRP: instances, solutions and code. https://github.com/felicze/3l-cvrp. MIT, commit 2b52d20.

[8] Felix Tamke, Florian Linß, Leopold Kuttner, and Udo Buscher. A branch-and-cut algorithm for vehicle routing problems with three-dimensional loading constraints. *European Journal of Operational Research*, 331:790–808, 2026.

[9] Thibaut Vidal. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research*, 140:105643, 2022.